Credit Card Fraud Detection System: Project Documentation

Nov 10, 2026 · @Novash Sharma

This project builds a machine learning model that flags credit card transactions likely to be fraud. This document explains what we are building, why, and how.

1. What We Are Building

We are building a **credit card fraud detection system**. The system learns from past transactions, where each one is labeled as fraud or normal. When a new transaction arrives, it estimates how likely that transaction is to be fraud.

Item	Details
Project type	Machine learning prototype for learning and portfolio use
Problem type	Binary classification (fraud or normal) on highly imbalanced data
Real-world user	A bank or payment company, running the model on its own servers
Data	Public Kaggle "Credit Card Fraud Detection" dataset, released by Worldline and ULB
Data size	284,807 transactions, about 0.17% of them fraud
Data fields	`V1` to `V28` (anonymized), `Time`, `Amount`, and the label `Class`
Personal data	None. All fields are anonymized, and no client data is used
Deliverables	A trained model, a comparison of models, and clean code with a README on GitHub

2. Why We Are Doing This

Business need

- Fraud causes direct financial loss to customers and banks.
- Checking every transaction by hand is not possible, so detection must be automatic.

- Too many false alarms upset genuine customers. A good model catches fraud and avoids wrong alerts.

Technical challenge

- Fraud is very rare, so a basic model can ignore it and still look good.
- A model that calls every transaction "normal" is 99.8% accurate but catches no fraud. Accuracy alone is misleading here.
- This makes the project a good example of imbalanced data, the right metrics, and data leakage.

Learning goals

- Understand a full machine learning pipeline, from raw data to explained results.
- Practice teamwork and collaboration on GitHub.
- Produce a project that can be shown in a portfolio.

Why public data

- Real client data cannot be used without permission and legal agreements, because of privacy and payment-security rules.
- Public anonymized data lets us practice the same techniques with no privacy risk.

3. How We Will Do It

3.1 Approach

1. **Explore the data (EDA):** understand the data and confirm the class imbalance.
2. **Prepare the data:** split into 80% training and 20% test data (keeping the same fraud share in both), and scale `Amount` and `Time`.
3. **Balance the classes:** use SMOTE, which creates synthetic fraud examples, on training data only.
4. **Train models:** Logistic Regression (baseline), XGBoost and Random Forest.
5. **Improve the best model:** tune its settings and choose a better decision threshold.
6. **Explain the model:** use SHAP to see which features drive its predictions.

3.2 How we measure success

- **PR-AUC (main metric):** one score that summarizes precision and recall across all thresholds.
- **Precision:** of the transactions flagged as fraud, how many really were.
- **Recall:** of all real fraud, how much we caught.

- **Accuracy** is not a main metric, because it is misleading on imbalanced data.
- Every model is compared with the Logistic Regression baseline.

3.3 Rules for reliable results

- **Avoid data leakage:** scaling and SMOTE use training data only, and the test data stays untouched.
- **Cross-validation:** SMOTE runs inside a Pipeline, so each fold stays clean.
- The decision threshold is chosen on training data. The test set is used only for the final check.

3.4 Tools

Python, pandas, scikit-learn, imbalanced-learn, XGBoost, SHAP, Jupyter Notebook and GitHub.

3.5 Team and ways of working

Role	Responsibility
Data and EDA owner	EDA, data split and scaling
Modeling owner	Build and compare models
Evaluation owner	Score every model the same way, tuning and threshold
Explainability and Docs owner	SHAP, README and documentation
Team Lead	Direction, code review and merging

Roles assume a team of four members and can be adjusted.

Working rules:

- Everyone uses the same split: stratified 80/20 with `random_state=42`.
- One main metric: PR-AUC.
- Each task is done on its own branch and merged into `main` after review.
- The dataset and saved model files (`.pkl`) are not uploaded to GitHub. Each member downloads the dataset from Kaggle.

4. Limitations

- This is a prototype built on public data from 2013.

- A real company would retrain the model on its own data, because fraud patterns change over time.
- The test set has few fraud cases, so small differences in scores should not be over-read.